C언어 6시간 압축 학습 PDF

재혁님용 6주차 준비 자료
목표: 오늘 6시간 동안 C언어의 핵심만 익혀서 `포인터`와 `Linked List` 구현을 시작할 수 있는 상태 만들기

이 자료를 보는 방법

이 자료는 C언어를 전부 배우기 위한 문서가 아닙니다.
오늘 필요한 것만 빠르게 익히고, 바로 자료구조 과제로 넘어가기 위한 문서입니다.

오늘의 큰 목표는 3가지입니다.

1. C 코드가 어떻게 생겼는지 익숙해지기
2. 포인터와 메모리 주소를 무서워하지 않게 되기
3. `malloc`, `free`, `struct`를 이해해서 Linked List를 시작할 준비 끝내기

6시간 학습 로드맵

시간	주제	목표
1시간차	C언어 기본 구조, 변수, 자료형	C 코드의 뼈대 읽기
2시간차	연산자, 조건문, 반복문	흐름 제어 이해하기
3시간차	함수, 배열, 문자열	코드 쪼개기와 데이터 다루기
4시간차	주소와 포인터	`&`, `*` 의미 잡기
5시간차	`struct`, `malloc`, `free`	노드 구조 준비하기
6시간차	Linked List 입문, 직접 테스트	과제 진입 준비 완료

---

0. 먼저 준비할 환경

이번 과제는 Ubuntu 22.04 LTS 이상 환경을 사용합니다.
하지만 오늘은 문법 공부가 목표이므로, 우선 아래 2가지만 알면 충분합니다.

- `gcc 파일이름.c -o 실행파일이름`
- `./실행파일이름`

예시:

```
gcc main.c -o main
./main
```

Windows에서 바로 하지 않아도 괜찮습니다.
오늘은 문법 이해가 우선이고, 실습은 Ubuntu 환경에서 이어서 해도 됩니다.

1시간차. C언어 기본 구조 익히기

1-1. C 프로그램의 가장 기본 모양

```
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

1-2. 꼭 알아야 하는 것

- `#include <stdio.h>` : `printf` 같은 기능을 쓰기 위해 불러오는 헤더
- `int main(void)` : 프로그램이 시작되는 함수
- `{}` : 코드 묶음
- `;` : 문장 끝
- `return 0;` : 프로그램이 정상 종료되었다는 뜻

1-3. 변수와 자료형

```
int age = 20;
char grade = 'A';
float height = 175.5f;
double weight = 70.25;
```

자주 보는 자료형:

- `int` : 정수
- `char` : 문자 1개
- `float` : 실수
- `double` : 더 정밀한 실수

1-4. 오늘 꼭 기억할 포인트

- C는 변수 선언을 명확하게 해야 합니다.
- 파이썬처럼 변수 타입이 자동으로 바뀌는 느낌이 아닙니다.
- 문자 하나는 `'A'`, 문자열은 `"ABC"`처럼 다릅니다.

1시간차 체크

- `main` 함수가 무엇인지 설명할 수 있는가
- `int`, `char`, `double` 차이를 대충 말할 수 있는가
- 세미콜론 `;`을 빼먹지 않는가

2시간차. 연산자, 조건문, 반복문

2-1. 기본 연산자

```
int a = 10;
int b = 3;

printf("%d\n", a + b);
printf("%d\n", a - b);
printf("%d\n", a * b);
printf("%d\n", a / b);
printf("%d\n", a % b);
```

2-2. 비교 연산자와 조건문

```c
int score = 85;

if (score >= 90) {
    printf("A\n");
} else if (score >= 80) {
    printf("B\n");
} else {
    printf("C\n");
}
```

2-3. 반복문

`for` 문:

```c
for (int i = 0; i < 5; i++) {
    printf("%d\n", i);
}
```

`while` 문:

```c
int i = 0;
while (i < 5) {
    printf("%d\n", i);
    i++;
}
```

2-4. 초보자가 자주 헷갈리는 부분

- `=` 는 대입
- `==` 는 같은지 비교
- `i++` 는 1 증가
- 조건식은 보통 참이면 실행, 거짓이면 건너뜁니다

2시간차 미니 연습

직접 해보세요.

1. 1부터 10까지 출력하기
2. 1부터 10까지 합 구하기
3. 짝수만 출력하기
4. 점수에 따라 A/B/C 출력하기

3시간차. 함수, 배열, 문자열

3-1. 함수

함수는 코드를 역할별로 나누는 도구입니다.

```
int add(int a, int b) {
    return a + b;
}
```

```
int result = add(3, 5);
printf("%d\n", result);
```

3-2. 배열

배열은 같은 자료형 여러 개를 연속으로 저장합니다.

```
int nums[5] = {10, 20, 30, 40, 50};

printf("%d\n", nums[0]);
printf("%d\n", nums[4]);
```

주의:

- 인덱스는 0부터 시작
- `nums[5]`는 범위를 벗어남

3-3. 문자열

C에서 문자열은 사실 `char` 배열입니다.

```
char name[] = "Jungle";
printf("%s\n", name);
```

문자 하나:

```
char ch = 'A';
```

문자열:

```
char str[] = "ABC";
```

3-4. 이 시간이 중요한 이유

자료구조 구현에서는 함수로 기능을 나누게 됩니다.

예를 들면:

- 노드 생성 함수
- 삽입 함수
- 삭제 함수
- 출력 함수

그래서 함수 개념을 지금 익혀 두면, 과제 코드 구조를 읽기가 훨씬 쉬워집니다.

3시간차 체크

- 함수의 입력과 출력 개념을 설명할 수 있는가
- 배열 인덱스가 0부터 시작한다는 점을 기억하는가
- 문자열이 `char` 배열이라는 말을 이해했는가

4시간차. 주소와 포인터

오늘의 핵심입니다.
재혁님이 오늘 꼭 잡아야 하는 가장 중요한 주제입니다.

4-1. 메모리 주소란?

변수는 메모리 어딘가에 저장됩니다.
그 저장된 위치를 주소라고 생각하면 됩니다.

```
int x = 10;
printf("%p\n", (void*)&x);
```

- `&x` : x의 주소
- `%p` : 주소 출력 형식

4-2. 포인터란?

포인터는 "주소를 저장하는 변수"입니다.

```
int x = 10;
int *p = &x;
```

이 뜻은:

- `x` 는 값 10을 저장
- `&x` 는 x의 주소
- `p` 는 그 주소를 저장

4-3. `*` 와 `&` 구분

```
int x = 10;
int *p = &x;
```

- `&x` : x의 주소를 가져옴
- `*p` : p가 가리키는 주소에 들어 있는 실제 값

예시:

```
int x = 10;
int *p = &x;

printf("%d\n", x);
printf("%p\n", (void*)p);
printf("%d\n", *p);
```

4-4. 왜 자료구조에서 포인터가 중요한가

Linked List 노드는 보통 이렇게 생깁니다.

- 데이터 하나
- 다음 노드의 주소 하나

즉, 다음 칸의 "값"을 저장하는 것이 아니라,
다음 칸이 어디 있는지 "주소"를 저장합니다.

그래서 포인터를 모르면 Linked List는 이해하기 어렵고,
포인터를 이해하면 Linked List는 "칸들을 주소로 연결한 구조"가 됩니다.

4-5. 포인터 연습 문제

직접 손으로 써보세요.

```
int a = 7;
int *p = &a;
```

아래 질문에 답해보세요.

1. `a` 의 값은 무엇인가
2. `&a` 는 무엇인가
3. `p` 에는 무엇이 저장되는가
4. `*p` 의 값은 무엇인가

4-6. 포인터에서 흔한 실수

- 포인터를 선언만 하고 초기화하지 않음
- `*p` 와 `p` 를 헷갈림
- 주소를 출력하면서 `%d` 를 사용함
- `NULL` 개념을 아직 모른 채 포인터를 마구 사용함

5시간차. struct, malloc, free

이제 자료구조의 재료를 배웁니다.

5-1. `struct`

서로 관련 있는 데이터를 하나로 묶고 싶을 때 사용합니다.

```
struct Student {
    int id;
    int score;
};
```

이제 `Student` 는 `id` 와 `score` 를 같이 가지는 구조가 됩니다.

5-2. 왜 `struct` 가 Linked List에서 필요한가

노드는 보통 데이터 하나만 있는 것이 아니라,

- 값
- 다음 노드 포인터

를 함께 가져야 합니다.
그래서 `struct` 가 필요합니다.

5-3. 동적 메모리 할당

실행 중에 필요한 만큼 메모리를 만들고 싶을 때 `malloc` 을 씁니다.

```
int *p = (int *)malloc(sizeof(int));
```

의미:

- `sizeof(int)` 만큼 메모리 요청
- 그 시작 주소를 `p` 에 저장

값을 넣을 수 있습니다.

```
*p = 100;
printf("%d\n", *p);
```

다 썼으면 반드시 반납합니다.

```
free(p);
```

5-4. 왜 `free` 가 중요한가

`malloc` 으로 빌린 메모리를 반납하지 않으면 메모리 누수 문제가 생깁니다.
자료구조 구현에서는 노드를 계속 만들고 지우므로, `free` 습관이 매우 중요합니다.

5-5. `NULL` 확인

`malloc` 이 실패할 수도 있으므로 보통 확인합니다.

```
if (p == NULL) {
    return 1;
}
```

5-6. 이 시간의 핵심 연결

이제 재혁님은 아래 흐름을 이해하면 됩니다.

1. `struct` 로 노드 모양 정의
2. `malloc` 으로 새 노드 생성
3. 포인터로 다음 노드 연결
4. 필요 없으면 `free` 로 해제

이게 Linked List의 뼈대입니다.

6시간차. Linked List 시작 전 개념 정리

6-1. 배열과 Linked List 차이

배열:

- 메모리가 연속적
- 인덱스로 빠르게 접근 가능
- 크기 변경이 불편함

Linked List:

- 노드들이 주소로 연결됨
- 중간 삽입/삭제가 유리함
- 인덱스 접근이 느림

6-2. 노드 개념

노드는 보통 아래 두 가지를 가집니다.

1. 현재 값
2. 다음 노드를 가리키는 포인터

그림으로 보면:

```
[10 | next] -> [20 | next] -> [30 | NULL]
```

6-3. Linked List에서 가장 먼저 구현할 기능

처음부터 모든 기능을 하려고 하지 말고, 아래 순서로 가세요.

1. 노드 하나 만들기
2. 노드 두 개 연결하기
3. 리스트 출력하기
4. 맨 앞 삽입
5. 맨 뒤 삽입

6. 값 찾기
7. 삭제

6-4. 초보자용 테스트 방법

테스트 케이스가 따로 없으니 직접 만들어야 합니다.

예시:

1. 빈 리스트에서 출력하기
2. 노드 1개만 넣고 출력하기
3. 3개 넣고 순서 확인하기
4. 맨 앞 삭제하기
5. 없는 값 삭제 시도하기
6. 마지막 노드 삭제 후 빈 리스트 되는지 확인하기

6-5. 재혁님이 과제에서 자주 막히는 지점

- 포인터가 가리키는 대상을 헷갈림
- `malloc`만 하고 `free`를 잊음
- `head`가 `NULL`인 경우를 놓침
- 삭제할 때 이전 노드와 다음 노드 연결을 잘못함
- 함수로 분리하지 않아 코드가 너무 복잡해짐

오늘 6시간 학습 후 도달해야 하는 상태

이 문서를 다 보고 나면 아래가 되어야 합니다.

- C 코드의 기본 형태를 읽을 수 있다
- 변수, 조건문, 반복문, 함수가 무엇인지 안다
- 배열과 문자열을 대충 다룰 수 있다
- `&`, `*`, 포인터의 의미를 설명할 수 있다
- `malloc`, `free`, `struct`가 왜 필요한지 안다
- Linked List가 주소로 연결된 구조라는 점을 이해한다

이 정도면 오늘 목표는 충분합니다.
아직 구현을 매끄럽게 못 해도 괜찮습니다.
오늘은 "익숙해지기 시작하는 날"이라고 생각하면 됩니다.

오늘 공부 체크리스트

아래 항목 중 체크가 많이 될수록 좋습니다.

- [ ] `main` 함수가 무엇인지 안다
- [ ] `if`, `for`, `while`을 읽을 수 있다
- [ ] 배열과 문자열 차이를 안다
- [ ] 함수의 입력과 출력을 이해한다
- [ ] `&x`와 `*p`의 차이를 설명할 수 있다
- [ ] 포인터가 주소를 저장하는 변수라는 말을 이해한다
- [ ] `malloc`은 메모리를 빌리는 것이라고 이해한다
- [ ] `free`는 메모리를 반납하는 것이라고 이해한다
- [ ] `struct`가 여러 데이터를 묶는 도구라는 점을 안다
- [ ] Linked List 노드가 값과 다음 주소를 가진다는 점을 안다

오늘 바로 보면 좋은 자료

- W3Schools C Tutorial
- `Syntax`
- `Variables`
- `Data Types`
- `if...else`
- `while`
- `for`
- `Functions`
- `Arrays`

- `Strings`
- `Memory Address`
- `Pointers`

이 순서까지만 봐도 오늘 목표에는 충분합니다.

6시간 세부 진행표

1교시 50분

- C 프로그램 기본 구조 보기
- 변수, 자료형, `printf` 익히기
- 직접 아주 짧은 코드 읽어보기

쉬는 시간 10분

2교시 50분

- 연산자
- 조건문
- 반복문
- 1부터 10 출력, 합 구하기 직접 해보기

쉬는 시간 10분

3교시 50분

- 함수
- 배열
- 문자열
- 함수로 계산 결과 출력해보기

쉬는 시간 10분

4교시 50분

- 메모리 주소
- 포인터
- `&`, `*` 연습
- 종이에 메모리 그림 그려보기

쉬는 시간 10분

5교시 50분

- `struct`
- `malloc`
- `free`
- `NULL`
- 동적 메모리 개념 정리

쉬는 시간 10분

6교시 50분

- Linked List 구조 그림 보기
- 노드 2개 연결되는 흐름 이해하기
- 직접 테스트 케이스 적어보기
- 내일 공부할 구현 순서 정리하기

내일 바로 이어서 할 것

오늘 문법 공부가 끝나면, 내일은 아래 순서로 가면 좋습니다.

1. 노드 구조 다시 보기
2. `head` 개념 이해하기
3. 노드 생성 함수 생각하기
4. 삽입 함수부터 구현하기
5. 출력 함수로 상태 확인하기
6. 삭제는 마지막에 하기

마지막 한 줄 정리

오늘 재혁님이 해야 할 가장 중요한 공부는 "C언어 전체"가 아니라 "`포인터`, `malloc/free`, `struct`를 이해해서 Linked List를 시작할 준비를 하는 것"입니다.